Typed Plans, Lightweight Judges: Rethinking Representation and Verification in Tool-Calling Agents

LMA Group Project Proposal

Team Members:
Akshat Pandey
Aryan Jain
Sarwari Mallela
Kotiya Mohith Goud

LMA — IIIT Hyderabad
Instructor: Prof. Vasudeva Varma

1. Domain and Target User

Our project focuses on **LLM-based tool calling agents** that decompose a natural-language user request into subtasks, then plan and choose the sequence of external tool or API invocations to serve that request, resolve dependencies between intermediate results, and execute that sequence to completion. The literature has moved from studying isolated, single-tool invocation toward long-horizon, multi-tool execution, where the central reliability challenge shifts from ”can the model call one tool correctly” to ”can it maintain correctness across an extended trajectory with intermediate state, execution feedback, and dependent steps.”

Domain: Domain-agnostic architecture. Can be used by any multi-agent system that delegates multi-step, tool-dependent tasks to such an agent, where a single incorrect tool call, broken dependency, or unverified intermediate result can propagate through the remainder of execution.

Target user: Users interacting with agents that must plan, execute, verify, and recover from multi-step tool-based tasks.

2. Problem Statement

LLM-based agents are increasingly used for tasks that require multiple tool calls rather than a single model response. However, an agent may generate a plausible-looking plan containing incorrect dependencies, incompatible intermediate outputs, missing steps, or inappropriate tool selections. Such errors become more costly once execution begins because an incorrect intermediate result can propagate to downstream operations and require previously successful computation to be repeated. Existing work has explored plan-level verification, inference-time feedback, and pre/post-execution evaluation, but these mechanisms address different parts of the execution lifecycle. Furthermore, invoking an expensive LLM-based judge after every tool call can increase inference cost and may introduce unnecessary modifications to correct computation. We therefore investigate an execution-aware architecture that introduces better planning, more determinism and reliability in the existing less reliable LLM based approahces.

3. Why the Problem Matters

The reliability of a tool-calling agent depends not only on selecting appropriate tools, but also on maintaining valid dependencies and intermediate states throughout execution. In a multi-step workflow, an incorrect output from one node can become the input to several downstream nodes, amplifying a local error into a trajectory-level failure. A recovery mechanism that simply restarts the entire trajectory also repeats tool calls whose results were already correct.

This motivates an **execution-aware** agent in which verification is part of the execution loop. The objective is not to apply the most expensive reasoning mechanism to every step, but to combine verification mechanisms with different computational costs and intervene only when evidence of a problem exists.

4. What Is Missing in Existing Approaches

Early tool calling agents follow the greedy approach, they reason, call a tool, observe and proceed (e.g. ReAct agents [5]), there's no plan verification at all, so errors surface only after they cause a downstream failure.

More recent approaches try to provide several components to improve on them, and try to shift towards more reliable tool-calling and planning. Works like **GNNVerifier** represents generated plans as attributed directed graphs and estimates graph-, node-, and edge-level risk to guide plan correction [4]. Its focus motivates explicit structural verification, but our system extends verification into execution, where actual tool outputs are available.

Reinforced Agent introduces inference-time feedback for provisional tool calls and demonstrates the value of separating execution from review [11]. It also highlights the trade-off between correcting errors and introducing new errors through intervention. This motivates selectively routing suspicious outcomes to a stronger LLM judge rather than reviewing every operation.

ToolTree combines pre-execution and post-execution evaluation within tool planning and uses feedback to guide search [13]. Our approach adopts the complementary pre-/post-execution principle but operates through an explicit intermediate representation (IR), wave-based execution, selective execution-time classification, local IR repair, and preservation of successful computation.

5. Research Questions

- **RQ1.** Does representing a plan as a typed intermediate representation (IR), rather than free-form text or a graph, improve the planning quality of tool-calling agents?
- **RQ2.** Can a text based classifcation models like JEV [12], Laya [3] classification model can improve reliability, reduce unnecessary LLM-based intervention and recomputation while preserving recovery performance?

6. Hypothesis

We hypothesize that representing a plan as a typed intermediate representation, rather than free-form text or an unstructured graph, will improve planning quality by making dependencies, argument bindings, and effect classifications explicit and checkable, rather than left implicit in the plan's surface form. Further we hypothesize that layered verification will improve task success by detecting structural errors before execution and execution errors after tool outputs become available. We further hypothesize that using JEV[12] as a selective gate for LLM-based intervention will reduce token usage, verify semantic structure, and LLM/API calls relative to indiscriminate LLM-based review, while local repair and caching will reduce unnecessary recomputation.

7. Proposed Idea and Expected Novelty

- **Explicit IR-based planning** to represent tools, actions, and dependencies before execution. A detailed example of an IR along with the user query is provided in fig.2.
- **Runtime verification** to check intermediate tool outcomes during execution.
- **Selective verification** where JEV [12](or its open-source alternative, Laya[3]) acts as a lightweight text-based classifier to identify whether an execution outcome is acceptable or requires further analysis. This is used to selectively invoke a larger LLM as a judge instead of using it for every tool outcome.
- **Local repair** to modify only the affected part of the IR when an execution error is detected.
- **Continued execution** by preserving valid computation and resuming from the repaired state.

Here, **JEV**[12] , or its open-source alternative **Laya**[3], is used as a text-based classifier. It takes the user query, available options, and relevant input/output information, and produces a probability distribution over the options. In our setting, it can classify an execution outcome as `OKAY`, `ERROR`, or `NEEDS_FORMAT_CHANGE`. The predicted outcome is then used to decide whether a larger LLM should be invoked for detailed verification or repair.

7.1 Execution Pipeline

1. **Planning:** An LLM converts the user query into an IR containing the required tools, actions, and dependencies.

2. **Compilation and pre-execution verification:** The IR is compiled into executable tool calls, and JEV[12] / Laya[3]checks it before execution. If an issue is found, the IR is revised and checked again.
3. **Wave-based execution:** Independent nodes are grouped into dependency-aware waves and executed in parallel. Example of wave wise execution for user query in fig.2 is given in fig.3 .
4. **Runtime verification:** After each wave, JEV[12] / Laya[3] classifies the observed tool outcomes.
5. **Selective intervention:** Only a problematic wave is passed to an LLM judge for deeper analysis using the IR, affected nodes, tools, and observed outputs.
6. **Local IR repair:** The judge can edit, remove, replace, or add affected nodes and update their dependencies.
7. **Recovery and caching:** Execution resumes from the repaired IR, while outputs of valid nodes are cached and reused whenever their dependencies remain unchanged.

The expected novelty lies in combining explicit IR-based planning, pre-execution verification, wave-level runtime checks, selective LLM reasoning, local IR repair, and reuse of verified computation in a continuous execution and recovery loop.

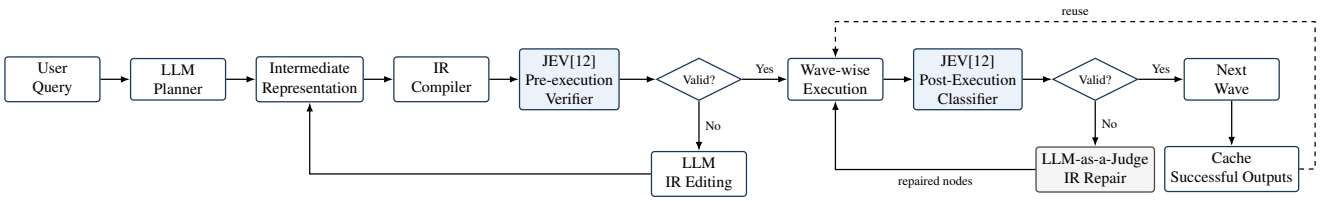


Figure 1: Execution-aware verification and recovery architecture. JEV[12] performs plan-level and execution-level classification. Plan-level failures are corrected by LLM IR editing and re-verified before execution, while execution-level failures are repaired by an LLM judge and only the affected nodes are re-executed, reusing cached successful outputs.

Request: *"Plan a Goa trip for 4 people: compare flight and train and book the cheaper one. Also find a beach hotel with free cancellation, and check the weather. If it is likely to rain, book an indoor activity, otherwise book a scooter. Then email everyone the itinerary."*

```
s1  = search_flights("BLR", "GOI")          → list of Flights    [safe]
s2  = search_trains("BLR", "GOI")           → list of Trains     [safe]
s3  = search_hotels(near="beach",
                    cancellation="free")    → list of Hotels     [safe]
s4  = get_weather("Goa", "weekend")         → Forecast           [safe]

s5  = pick_cheapest(s1 + s2)                → one Option         [safe]
s6  = check(s4.rain_chance > 0.5)           → yes/no             [safe]

s7  = book_transport(s5.id)                 → Booking            [DANGER]
s8  = book_hotel(s3.first.id)               → Booking            [DANGER]
s9  = book_indoor_activity()  if s6         → Booking            [DANGER]
s10 = book_scooter(people=4)  if not s6     → Booking            [DANGER]

s11 = send_email(s7, s8, s9 or s10)         → Sent               [DANGER]
```

Figure 2: The intermediate representation (IR) for the Goa trip request, written once as a sequence of typed steps. Each step declares its expression, return type, and safety tag (*safe* vs. *DANGER*); dependencies are expressed only through `sN` references, with no explicit instruction about execution order or parallelism.

8. Required Agent Capabilities

The proposed system combines the five required capabilities, with different levels of dependence on each. Reasoning, agentic execution, and tool calling form the core of the system, while RAG and fine-tuning support domain grounding and efficient model adaptation.

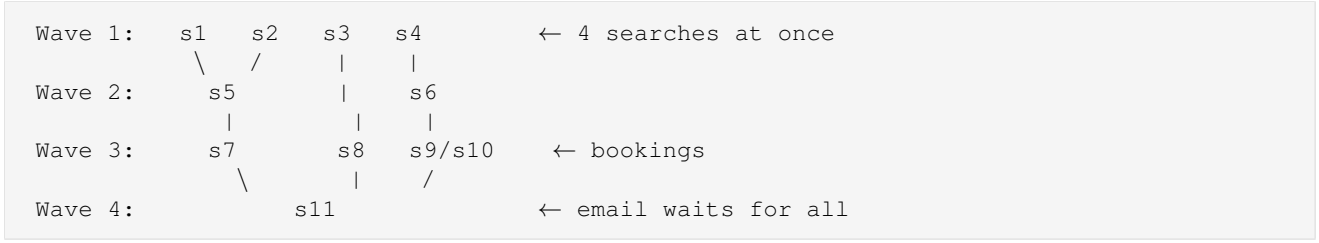


Figure 3: Wave-based execution of the IR in Figure 2. The compiler asks one question of every step: *are all the steps I need already finished?* Every step that answers yes at the same moment forms one wave, and a wave executes all at once. Wave 1 runs the four independent searches in parallel; Wave 3 executes the *DANGER*-tagged bookings; Wave 4 waits for every upstream booking before the email is sent.

Capability	**Why the problem needs it**	**How it will be measured**
Fine-tuning	LoRA/QLoRA will be used to adapt the IR generator LLM so that it can produce structured and executable intermediate representations.	IR quality, Task Success Rate, token usage, and inference cost.
Reasoning	The system needs to reason across multiple steps, including planning, checking dependencies, identifying failures, and deciding how the IR should be modified.	IR quality, recovery success, verification accuracy, and Task Success Rate.
Agentic Abilities	The agent needs to keep track of its execution state, check intermediate results, respond to failures, repair the IR when needed, and continue the task.	Task Success Rate, recovery success, API calls, tokens, and successful continuation after failures.
Tool Calling	The system needs to choose the right tools, generate valid arguments, follow tool dependencies, execute them, and handle incorrect or unexpected results.	Task Success Rate, tool calls, Unsafe Action Detection Rate, and False Intervention Rate.

Table 1: Agent capabilities relevant to the proposed system and their evaluation.

LoRA/QLoRA and the specific model configurations will be finalized during implementation. The capabilities and evaluation measures listed above reflect the current plan and may be adjusted as the experiments progress.

9. Dataset and Evaluation Environment

Evaluation will use established tool-calling benchmarks, an IR dataset constructed from existing tool-use data, and a controlled execution environment.

Evaluation benchmarks: The planned benchmark suite includes **τ^2-Bench [1], NESTFUL [2], BFCL [7], and TaskBench [10]**. These will be used to evaluate tool selection, planning, execution, and overall task success.

IR dataset: ToolACE[6] and NVIDIA's **When2Call**[9] will be used as sources for constructing the IR dataset. User queries and their associated tool information will be converted into IRs using a larger LLM, such as **Qwen**[8]. The model will receive the user query, the complete tool list, an example IR, and the rules defining a valid IR. The resulting IRs will form the training and evaluation data for the IR-based planning component.

Execution environment: The generated IRs will be executed in a sandboxed environment with mock tools. Controlled failures will be introduced during execution to evaluate error detection, intermediate verification, IR repair, and recovery.

IR evaluation: Since there is no ground-truth IR, a stratified subsample of the generated data will be manually evaluated by the four team members using a predefined rubric covering correctness and executability.

The benchmarks, model configuration, and evaluation settings may change during the course of the project based on the

experimental results.

10. Baselines

The primary comparisons will be against the three closely related systems motivating this work:

- **GNNVerifier** — structural graph-based plan verification and local correction [4];
- **Reinforced Agent** — inference-time feedback for tool-calling agents [11];
- **ToolTree** — pre/post-execution evaluation within tool planning and search [13].

Where applicable, their reported or reproducible benchmark settings will be used for comparison. A direct tool-calling agent using the same planning model and tools, but without verification or recovery, will serve as an additional simple baseline. Component ablations will remove pre-execution verification, execution-time classification, selective LLM intervention, or caching to isolate the contribution of each component.

11. Metrics

Evaluation will measure both **task-level reliability** and the **cost and safety of intervention**.

11.1 Capability-Specific Metrics

- **Success Rate (SR):** fraction of tasks completed correctly.
- **Average Tokens:** average number of tokens consumed during agent execution, LLM usage accounted for separately where applicable.
- **Average API Calls:** average number of model/API calls, reported separately for JEV[12] and the LLM judge.
- **IR Human-Evaluation Score:** rubric-based assessment of the correctness, completeness, dependency structure, and executability of generated IRs.
- **JEV [12] Classification Evaluation:** Accuracy, precision, recall, and F1-score for classification of whether an executed node is correct or incorrect.

These metrics allow us to evaluate the central trade-off between reliability and intervention cost rather than measuring task success alone.

11.2 Safety and Guardrail Metrics

Two primary guardrail metrics will be used:

- **Unsafe Action Detection Rate:** proportion of injected invalid or unsafe tool actions detected before they are executed.
- **False Intervention Rate:** proportion of valid actions that are incorrectly blocked, modified, or routed for unnecessary intervention.

These metrics evaluate both sides of the guardrail objective: detecting problematic actions while avoiding unnecessary interference with correct execution.

12. Analysis and Ablation Plan

The evaluation will examine where the architecture provides its gains and where failures remain. Analysis will focus on:

1. the ability of JEV[12] to detect structural and execution-time problems;
2. the extent to which selective gating reduces LLM calls and token usage;
3. the success and side effects of LLM-based IR repair;
4. the amount of valid computation preserved through caching; and
5. the contribution of individual components through ablations.

In particular, comparisons with and without JEV[12] gating, execution-time verification, LLM repair, and caching will distinguish reliability gains from efficiency gains.

13. Compute Plan

The project is primarily an inference-time system and therefore does not require large-scale model training. The current resource estimate is:

Resource	Planned Usage
GPU compute	∼100 GPU hours
Maximum API budget	∼$30
Primary API cost	JEV[12] and closed-source language model inference
Candidate/test models	GPT-4o mini, GPT-5.6 Luna, GPT-6 Luna Pro

Table 2: Tentative computational budget.

The candidate models listed above represent models that may be tested during experimentation and are not assumed to be the final model selection. The project will preferentially use open-source components where feasible, with the API budget primarily covering JEV [12] and closed-source model usage.

14. Expected Risks and Fallback Plan

Verification errors. JEV[12] may incorrectly reject valid plans or accept invalid ones. **Fallback:** evaluate false-positive and false-negative behavior separately and tune confidence thresholds or fallback policies.

Recovery propagation. A repair may invalidate downstream dependencies. **Fallback:** recompute only the affected dependency region while preserving cached outputs whose dependencies remain valid.

Benchmark or prototype limitations. Controlled environments may not capture every real-world tool failure. **Fallback:** evaluate across multiple established benchmarks and controlled failure settings, while clearly reporting the limits of generalization.

References

[1] Victor Barres, Hanlong Dong, Sanjay Ray, Xian Si, and Karthik Narasimhan. Evaluating conversational agents in a dual-control environment, 2025.

[2] Kinjal Basu, Ibrahim Abdelaziz, Kiran Kate, Mayank Agarwal, Maxwell Crouse, Yara Rizk, Kelsey Bradford, Asim Munawar, Sadhana Kumaravel, Saurabh Goyal, Xin Wang, Luis A. Lastras, and Pavan Kapanipathi. NESTFUL: A benchmark for evaluating LLMs on nested sequences of API calls. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025.

[3] ConvAI Innovations. Layab. https://huggingface.co/convaiinnovations/layab. Hugging Face Model Repository.

[4] Yu Hao, Qiuyu Wang, Cheng Yang, Yawen Li, Zhiqiang Zhang, and Chuan Shi. GNNVerifier: Graph-based verifier for LLM task planning, 2026.

[5] Jinchao Hu, Meizhi Zhong, Kehai Chen, Xuefeng Bai, and Min Zhang. Agentic tool use in large language models, 2026.

[6] Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dayiheng Li, Shihang Wang, et al. ToolACE: Winning the points of LLM function calling, 2024.

[7] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the Forty-Second International Conference on Machine Learning*, 2025.

[8] Qwen Team. Qwen3.8-Max: A new bar for coding and cowork, August 2026.

[9] Hayley Ross, Aditya S. Mahabaleshwarkar, and Yoshi Suhara. When2Call: When (not) to call tools. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3391–3409, 2025.

[10] Yongliang Shen, Kaitao Song, Xu Tan, Wenqi Zhang, Kan Ren, Siyu Yuan, Weiming Lu, Dongsheng Li, and Yueting Zhuang. TaskBench: Benchmarking large language models for task automation. *Advances in Neural Information Processing Systems*, 37:4540–4574, 2024.

[11] A. Ta, J. Zhu, and S. Shayandeh. Reinforced agent: Inference-time feedback for tool-calling agents, 2026.

[12] TypeSafe AI. Introducing system one models and JEV. https://typesafe.ai/blog/introducing-system-one-models-and-jev. TypeSafe AI Blog.

[13] S. Yang, S. C. Han, Y. Ding, S. Wang, and E. Hovy. ToolTree: Efficient LLM agent tool planning via dual-feedback monte carlo tree search and bidirectional pruning. In *International Conference on Learning Representations (ICLR)*, 2026.